EXPERIMENT NO. 8
COMPARATIVE STUDY OF OPENMP AND MPI FOR PARALLEL APPLICATIONS

HIGH PERFORMANCE COMPUTING (HPC)

Student information	Details
Name	Zaid Pathan
PRN	23510029

Important note about experimental readings

This report includes a complete implementation guide, six source-code listings, result tables, and a worked example dataset so that calculations and graph preparation are demonstrated. The numerical readings below are ILLUSTRATIVE SAMPLE READINGS, not measurements from Zaid's computer. They must not be represented as personally observed results. Run the programs on the lab system and replace the sample values with the actual output before final submission. Performance depends on CPU, compiler, MPI implementation, input size, and system load.

1. Aim

To implement matrix–matrix multiplication and two-dimensional convolution using sequential execution, OpenMP and MPI, and compare execution time, speedup, efficiency, scalability, memory models and communication overhead.

2. Objectives

- Implement the same computational problem using sequential, shared-memory and distributed-memory approaches.
- Measure execution time for multiple thread/process counts and input sizes.
- Calculate speedup and parallel efficiency.
- Study OpenMP scheduling and MPI data distribution/communication.
- Compare the suitability of OpenMP, MPI and hybrid MPI + OpenMP for HPC workloads.

3. System setup and tools

Parameter	Record for actual run
Operating system	Ubuntu Linux
CPU	Intel Core i5-1235U
Logical CPUs	12
RAM	Record using free -h
C++ compiler	g++ --version
MPI implementation	mpic++ --version / mpirun --version
OpenMP	GCC OpenMP support; compile with -fopenmp
Timing	std::chrono::steady_clock

4. Theory and formulas

4.1 OpenMP

OpenMP is a shared-memory parallel programming API. Threads execute within one process and can access shared arrays. Directives such as #pragma omp parallel for distribute loop iterations. Scheduling policies include static, dynamic and guided. Synchronization is needed when multiple threads might update shared state.

4.2 MPI

MPI (Message Passing Interface) supports distributed-memory parallel programming. Each process has its own address space. Data is distributed explicitly and processes communicate using point-to-point or collective operations such as MPI_Bcast, MPI_Scatterv and MPI_Gatherv. MPI can run across multiple compute nodes.

4.3 Metrics

Speedup $S_p = T_1 / T_p$, where T_1 is the sequential time and T_p is the parallel time using p threads/processes. Efficiency $E_p = (S_p / p) \times 100\%$. For comparisons, use the same input, algorithm, compiler optimization and timing scope. State whether setup, communication and output collection are included.

5. Implementation steps

1. Create a directory named experiment8 and save the six code listings below into separate files with the exact filenames shown.
2. Install tools if needed: sudo apt update && sudo apt install build-essential openmpi-bin libopenmpi-dev.
3. Compile sequential programs using g++ -O2 -std=c++17 filename.cpp -o program.
4. Compile OpenMP programs using g++ -O2 -std=c++17 -fopenmp filename.cpp -o program.
5. Compile MPI programs using mpic++ -O2 -std=c++17 filename.cpp -o program.
6. Run each program on the same input size. OpenMP uses OMP_NUM_THREADS=2, 4, 8; MPI uses mpirun -np 2, 4, 8.
7. Record the execution time printed by each program. Run each configuration at least three times and use the median or average consistently.
8. Check output correctness against the sequential version before comparing speed.
9. Calculate speedup and efficiency, then create the four required graph types for each application.

6. Task 1 — Matrix–matrix multiplication

For square matrices A and B of size N × N, compute C = A × B, where C[i][j] = Σ(k=0 to N−1) A[i][k] × B[k][j]. The OpenMP version parallelizes output rows. The MPI version partitions rows of A and C; B is broadcast to all ranks.

6.1 Run commands

Sequential

```
g++ -O2 -std=c++17 matmul_seq.cpp -o matmul_seq
./matmul_seq 500
./matmul_seq 1000
```

OpenMP

```
g++ -O2 -std=c++17 -fopenmp matmul_omp.cpp -o matmul_omp
OMP_NUM_THREADS=2 ./matmul_omp 1000
OMP_NUM_THREADS=4 ./matmul_omp 1000
OMP_NUM_THREADS=8 ./matmul_omp 1000
```

MPI

```
mpic++ -O2 -std=c++17 matmul_mpi.cpp -o matmul_mpi
mpirun -np 2 ./matmul_mpi 1000
mpirun -np 4 ./matmul_mpi 1000
mpirun -np 8 ./matmul_mpi 1000
```

6.2 Sample readings — illustrative only

Example workload: N = 1000. These values are fabricated for demonstrating the report format and calculations; replace them with program output.

Model	Threads / ranks	Time (s)	Speedup	Efficiency (%)
Sequential	1	2.400	1.00	100.0
OpenMP	2	1.330	1.80	90.2
OpenMP	4	0.720	3.33	83.3
OpenMP	8	0.510	4.71	58.9
MPI	2	1.520	1.58	78.9
MPI	4	0.900	2.67	66.7
MPI	8	0.680	3.53	44.1

Example calculation for OpenMP with 4 threads: speedup = 2.400 / 0.720 = 3.33; efficiency = (3.33 / 4) × 100 = 83.3%.

6.3 Discussion

In this illustrative dataset, OpenMP is faster than MPI on one shared-memory machine because the MPI version adds process coordination and data movement. As workers increase, speedup is sublinear due to memory bandwidth, cache effects, synchronization, serial work and overhead. Actual rankings may differ on another machine or for another implementation.

7. Task 2 — 2D convolution

Convolution applies a small kernel to an input grid. This implementation uses a 3 × 3 kernel of ones and computes the sum of the nine neighboring values for interior pixels. Border output cells are set to zero. The same policy is used in all three implementations.

7.1 Run commands

Sequential

```
g++ -O2 -std=c++17 conv_seq.cpp -o conv_seq
./conv_seq 1000
```

OpenMP

```
g++ -O2 -std=c++17 -fopenmp conv_omp.cpp -o conv_omp
OMP_NUM_THREADS=2 ./conv_omp 1000
OMP_NUM_THREADS=4 ./conv_omp 1000
OMP_NUM_THREADS=8 ./conv_omp 1000
```

MPI

```
mpic++ -O2 -std=c++17 conv_mpi.cpp -o conv_mpi
mpirun -np 2 ./conv_mpi 1000
mpirun -np 4 ./conv_mpi 1000
mpirun -np 8 ./conv_mpi 1000
```

7.2 Sample readings — illustrative only

Example workload: 1000 × 1000 grid. Replace these illustrative values with actual timings from your system.

Model / schedule	Threads / ranks	Time (s)	Speedup	Efficiency (%)
Sequential	1	0.950	1.00	100.0
OpenMP static	2	0.560	1.70	84.8
OpenMP static	4	0.310	3.06	76.6
OpenMP static	8	0.210	4.52	56.5
MPI	2	0.690	1.38	68.8
MPI	4	0.420	2.26	56.5
MPI	8	0.330	2.88	36.0

Convolution has a relatively small amount of arithmetic per input cell. MPI communication and setup can therefore be a noticeable fraction of runtime. The supplied MPI code broadcasts the full input grid to each rank for simplicity, rather than exchanging only halo rows; this is correct for the demonstration but uses more communication and memory than a carefully optimized halo-exchange implementation.

8. Performance graph requirements

- Graph 1: Number of threads/processes vs execution time — plot OpenMP and MPI separately for each application.
- Graph 2: Number of threads/processes vs speedup.
- Graph 3: Number of threads/processes vs efficiency.
- Graph 4: Problem size vs execution time — compare sequential, OpenMP and MPI.
- Label axes with units; mention input size and use actual measured data. Do not graph the illustrative readings as if they were measured.

9. OpenMP vs MPI architecture comparison

Feature	OpenMP	MPI
Memory model	Shared memory	Distributed memory
Execution unit	Thread	Process / rank
Communication	Shared variables/memory	Explicit message passing
Synchronization	Barrier, critical, atomic, reduction	Send/receive and collectives
Data distribution	Threads share arrays; work assigned by scheduling	Program partitions and sends data to ranks
Typical overhead	Often lower within one node	Communication and coordination costs

Scaling	Within shared-memory system	Across processes and nodes
Multi-node	Not sufficient alone for general distributed data	Well suited
Complexity	Often simpler for loop parallelism	Requires explicit data distribution and communication

10. Hybrid MPI + OpenMP challenge

For four nodes with eight cores per node, a conceptual mapping is one MPI rank per node with eight OpenMP threads per rank, giving four ranks and up to 32 threads in total. MPI distributes matrix blocks across nodes; OpenMP threads compute each rank's local rows or tiles. Hybrid execution may reduce the number of MPI ranks and use shared memory within each node. Challenges include thread affinity, load balance, synchronization, memory bandwidth and avoiding oversubscription.

11. AI/HPC case study — Medical image processing

Medical image processing involves large CT, MRI or microscopy images and may include denoising, filtering, edge detection, segmentation and feature extraction. Many pixel-level operations perform similar calculations independently, making them suitable for data parallelism. On a multicore workstation, OpenMP can distribute image rows or tiles across threads that share the image arrays. Static scheduling is often suitable when each row has similar work, while dynamic or guided scheduling may help if the amount of work varies by region.

MPI is useful when datasets exceed the memory capacity of one node or when many images are processed across a cluster. Each rank can process a separate image or a partition of one large image. For neighborhood operations such as convolution, boundary pixels require values from neighboring partitions; ranks may exchange halo regions before computing outputs near the boundary. Communication volume and frequency affect scalability. A hybrid approach can use MPI to distribute images or tiles across nodes and OpenMP to process each node's local data. The best choice depends on image size, available memory, communication cost and measured performance.

12. Pre-lab and reflection questions

What is the difference between a thread and an MPI process?

Threads share a process address space; MPI processes generally have separate address spaces.

What is shared-memory parallelism?

Multiple threads work on data accessible in a common memory space.

What is distributed-memory parallelism?

Independent processes own separate memory and exchange required data explicitly.

How does OpenMP distribute loop iterations?

A work-sharing loop assigns iterations to threads according to its schedule.

How does MPI distribute data?

The program partitions data and uses communication operations such as scatter/broadcast/gather.

What is communication overhead?

Time spent transferring data and coordinating workers rather than computing the main result.

Why can MPI be slower for some problems?

Startup, communication and synchronization can outweigh computation for small workloads.

Why is MPI suitable for multiple nodes?

It supports processes with separate memory on different nodes.

What is strong scaling?

Reducing runtime for a fixed problem size as resources increase.

What is weak scaling?

Increasing problem size with resources while keeping work per resource approximately constant.

Why can both models implement the same algorithm?

The computation can be divided among workers whether they share memory or exchange messages.

Why is matrix multiplication parallelizable?

Different output rows/elements can be calculated independently once the input matrices are available.

Why does convolution need boundary handling in MPI?

Neighboring input rows may be required to calculate output cells near a partition edge.

What is a halo region?

A copy of neighboring boundary data used to calculate outputs at partition boundaries.

Why can scaling stop improving?

Hardware limits, memory bandwidth, synchronization, serial work and communication constrain speedup.

How should the programming model be selected?

Consider architecture, workload size, memory capacity, data dependencies, communication and measured results.

13. Conclusion

The experiment compares sequential, OpenMP and MPI approaches for matrix multiplication and 2D convolution. OpenMP uses shared-memory threads and is convenient for loop parallelism on a multicore node. MPI distributes work among independent processes and is suitable for multi-node systems, but introduces explicit data distribution and communication. Final conclusions must be updated using actual measured timings and graphs. Discuss the best-performing configuration for each workload and explain differences using the system architecture and overheads.

14. Appendix — Complete source code

Save each code listing into the filename stated in its heading. These are compact teaching implementations intended for correctness and timing comparisons, not peak-performance kernels.

A1. Sequential matrix multiplication — matmul_seq.cpp

```
// matmul_seq.cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <chrono>
using namespace std;
int main(int argc,char** argv){
 int n = (argc>1)?atoi(argv[1]):500;
 vector<double>A((size_t)n*n,1.0), B((size_t)n*n,1.0), C((size_t)n*n,0.0);
 auto start=chrono::steady_clock::now();
 for(int i=0;i<n;i++) for(int k=0;k<n;k++){
   double a=A[(size_t)i*n+k];
   for(int j=0;j<n;j++) C[(size_t)i*n+j]+=a*B[(size_t)k*n+j];
 }
 auto end=chrono::steady_clock::now();
```

```
 cout<<"N="<<n<<" time_s="<<chrono::duration<double>(end-start).count()
     <<" check="<<C[0]<<"\\n";
}
```

A2. OpenMP matrix multiplication — matmul_omp.cpp

```
// matmul_omp.cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <chrono>
#include <omp.h>
using namespace std;
int main(int argc,char** argv){
 int n=(argc>1)?atoi(argv[1]):500;
 vector<double>A((size_t)n*n,1.0),B((size_t)n*n,1.0),C((size_t)n*n,0.0);
 auto start=chrono::steady_clock::now();
 #pragma omp parallel for schedule(static)
 for(int i=0;i<n;i++) for(int k=0;k<n;k++){
   double a=A[(size_t)i*n+k];
   for(int j=0;j<n;j++) C[(size_t)i*n+j]+=a*B[(size_t)k*n+j];
 }
 auto end=chrono::steady_clock::now();
 cout<<"N="<<n<<" threads="<<omp_get_max_threads()
     <<" time_s="<<chrono::duration<double>(end-start).count()
     <<" check="<<C[0]<<"\\n";
}
```

A3. MPI matrix multiplication — matmul_mpi.cpp

```
// matmul_mpi.cpp
#include <mpi.h>
#include <iostream>
#include <vector>
#include <cstdlib>
using namespace std;
int main(int argc,char** argv){
 MPI_Init(&argc,&argv);
 int rank,size; MPI_Comm_rank(MPI_COMM_WORLD,&rank); MPI_Comm_size(MPI_COMM_WORLD,&size);
 int n=(argc>1)?atoi(argv[1]):500;
 vector<int> rows(size), counts(size), displs(size), acount(size), adispls(size);
 int base=n/size, rem=n%size, off=0;
 for(int r=0;r<size;r++){ rows[r]=base+(r<rem); counts[r]=rows[r]*n;
   displs[r]=off*n; acount[r]=rows[r]*n; adispls[r]=off*n; off+=rows[r]; }
 vector<double>A, B((size_t)n*n,1.0), C;
 if(rank==0){A.assign((size_t)n*n,1.0); C.resize((size_t)n*n);}
 vector<double> localA((size_t)rows[rank]*n), localC((size_t)rows[rank]*n,0.0);
 MPI_Barrier(MPI_COMM_WORLD); double t0=MPI_Wtime();
 MPI_Bcast(B.data(),n*n,MPI_DOUBLE,0,MPI_COMM_WORLD);
 MPI_Scatterv(rank==0?A.data():nullptr,counts.data(),displs.data(),MPI_DOUBLE,
              localA.data(),counts[rank],MPI_DOUBLE,0,MPI_COMM_WORLD);
 for(int i=0;i<rows[rank];i++) for(int k=0;k<n;k++){
   double a=localA[(size_t)i*n+k];
   for(int j=0;j<n;j++) localC[(size_t)i*n+j]+=a*B[(size_t)k*n+j];
 }
 MPI_Gatherv(localC.data(),counts[rank],MPI_DOUBLE,
             rank==0?C.data():nullptr,counts.data(),displs.data(),MPI_DOUBLE,0,MPI_COMM_WORLD);
 double elapsed=MPI_Wtime()-t0, maxElapsed=0;
 MPI_Reduce(&elapsed,&maxElapsed,1,MPI_DOUBLE,MPI_MAX,0,MPI_COMM_WORLD);
 if(rank==0) cout<<"N="<<n<<" ranks="<<size<<" time_s="<<maxElapsed<<" check="<<C[0]<<"\\n";
 MPI_Finalize();
}
```

A4. Sequential 2D convolution — conv_seq.cpp

```
// conv_seq.cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <chrono>
using namespace std;
int main(int argc,char** argv){
 int n=(argc>1)?atoi(argv[1]):1000;
 vector<double>I((size_t)n*n,1.0),O((size_t)n*n,0.0);
 auto t0=chrono::steady_clock::now();
```

```
 for(int i=1;i<n-1;i++) for(int j=1;j<n-1;j++){
   double s=0;
   for(int u=-1;u<=1;u++) for(int v=-1;v<=1;v++)
     s+=I[(size_t)(i+u)*n+(j+v)];
   O[(size_t)i*n+j]=s;
 }
 auto t1=chrono::steady_clock::now();
 cout<<"N="<<n<<" time_s="<<chrono::duration<double>(t1-t0).count()
     <<" check="<<O[(size_t)(n/2)*n+n/2]<<"\\n";
}
```

A5. OpenMP 2D convolution — conv_omp.cpp

```
// conv_omp.cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <chrono>
#include <omp.h>
using namespace std;
int main(int argc,char** argv){
 int n=(argc>1)?atoi(argv[1]):1000;
 vector<double>I((size_t)n*n,1.0),O((size_t)n*n,0.0);
 auto t0=chrono::steady_clock::now();
 #pragma omp parallel for schedule(static)
 for(int i=1;i<n-1;i++) for(int j=1;j<n-1;j++){
   double s=0;
   for(int u=-1;u<=1;u++) for(int v=-1;v<=1;v++)
     s+=I[(size_t)(i+u)*n+(j+v)];
   O[(size_t)i*n+j]=s;
 }
 auto t1=chrono::steady_clock::now();
 cout<<"N="<<n<<" threads="<<omp_get_max_threads()
     <<" time_s="<<chrono::duration<double>(t1-t0).count()
     <<" check="<<O[(size_t)(n/2)*n+n/2]<<"\\n";
}
```

A6. MPI 2D convolution — conv_mpi.cpp

```
// conv_mpi.cpp
// Simple correct baseline: broadcast the full input to each rank, partition output rows.
#include <mpi.h>
#include <iostream>
#include <vector>
#include <cstdlib>
using namespace std;
int main(int argc,char** argv){
 MPI_Init(&argc,&argv);
 int rank,size; MPI_Comm_rank(MPI_COMM_WORLD,&rank); MPI_Comm_size(MPI_COMM_WORLD,&size);
 int n=(argc>1)?atoi(argv[1]):1000;
 vector<int> rows(size),counts(size),displs(size); int base=n/size,rem=n%size,off=0;
 for(int r=0;r<size;r++){rows[r]=base+(r<rem);counts[r]=rows[r]*n;displs[r]=off*n;off+=rows[r];}
 vector<double>I((size_t)n*n,1.0), localO((size_t)rows[rank]*n,0.0), O;
 if(rank==0) O.resize((size_t)n*n);
 MPI_Barrier(MPI_COMM_WORLD); double t0=MPI_Wtime();
 MPI_Bcast(I.data(),n*n,MPI_DOUBLE,0,MPI_COMM_WORLD);
 int first=displs[rank]/n;
 for(int li=0;li<rows[rank];li++){
   int i=first+li;
   if(i==0||i==n-1) continue;
   for(int j=1;j<n-1;j++){
     double s=0;
     for(int u=-1;u<=1;u++) for(int v=-1;v<=1;v++)
       s+=I[(size_t)(i+u)*n+(j+v)];
     localO[(size_t)li*n+j]=s;
   }
 }
 MPI_Gatherv(localO.data(),counts[rank],MPI_DOUBLE,
             rank==0?O.data():nullptr,counts.data(),displs.data(),MPI_DOUBLE,0,MPI_COMM_WORLD);
 double elapsed=MPI_Wtime()-t0,maxElapsed=0;
 MPI_Reduce(&elapsed,&maxElapsed,1,MPI_DOUBLE,MPI_MAX,0,MPI_COMM_WORLD);
 if(rank==0) cout<<"N="<<n<<" ranks="<<size<<" time_s="<<maxElapsed
                 <<" check="<<O[(size_t)(n/2)*n+n/2]<<"\\n";
```

```
 MPI_Finalize();
}
```

Appendix B — Useful system commands

System and compiler information

```
lscpu
free -h
g++ --version
mpic++ --version
mpirun --version
```

Appendix C — Final pre-submission checklist

- Replace every illustrative timing with actual measured values from the lab system.
- Recalculate speedup and efficiency from actual timings.
- Insert graphs generated from the actual readings.
- Record OS, CPU, RAM, compiler and MPI version.
- Run all six implementations and verify their check values match for the same input.
- Include source files with the DOCX if required by the instructor.